

## 4. Retrospective

This section reflects on the project development process, lessons learned, and future improvements.

### What Went Well [OK]

#### Technical Successes

- \* The backend architecture (controllers ? services ? repositories) with Prisma + PostgreSQL provided a clear structure for read-heavy endpoints and helped keep API behavior predictable.
- \* Docker + Docker Compose made local development reproducible (API + DB + Redis), which simplified troubleshooting and allowed the project to be run consistently across machines.
- \* The normalized data model with translation tables (EN/RU) and junction tables for symptoms/risk factors supported the project?s main academic goal: standardized ?disease cards? with explicit primary vs secondary factors.
- \* Observability was set up early (Winston/Morgan on the backend and Sentry hooks), which helped reduce time spent debugging silent failures.
- \* Testing infrastructure exists on both sides (backend unit tests with Jest; frontend unit tests + Playwright e2e), which provides a foundation for improving reliability.

#### Process Successes

- \* The scope was kept focused on a read-only MVP (no user accounts, no PHI/PII processing), which reduced compliance risk and kept the system design defensible for a thesis project.
- \* Documentation was treated as a deliverable: API reference, DB schema, criteria/ADR-style pages, and the user guide were aligned to the actual codebase instead of staying as templates.
- \* Incremental development worked well: backend/data model + docs were stabilized first, then the user-facing flows were documented and validated against repository reality.

#### Personal Achievements

- \* Built and maintained an end-to-end stack (React + Vite frontend, Express + Prisma backend, PostgreSQL + Redis, Docker Compose) within a thesis timeline.
- \* Improved skills in data modeling and schema evolution (Prisma migrations, seed strategy, normalized entities + translations).
- \* Strengthened the ability to translate academic requirements into an implementable MVP (scope control, out-of-scope decisions, and criterion-based documentation).

### What Didn't Go As Planned [!]

| Planned                                                                           | Actual Outcome                                               | Cause                                                             | Impact |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------|--------|
| Fully working frontend pages for the MVP                                          | Frontend is partially scaffolded; some pages are missing     | Small amount of time and the technical complexity of the frontend | Medium |
| Standardized ?Disease Card? page (UI + data model support exists in the codebase) | Page (UI + data model support exists in the codebase)        | Time constraints; prioritization of other features                | High   |
| Fully functional localization toggle                                              | Locale support exists in the backend but not in the frontend | Limited time for implementing full localization                   | Medium |

## Challenges Encountered

- \* **Late / shifting requirements for frontend delivery** - Problem: Frontend requirements were clarified late, after the planned deadline. - Impact: The UI could not be completed to the same level as the backend/data model and documentation, and some routes remain scaffolded rather than finished. - Resolution: Focused on stabilizing the backend, database schema, and documentation so the project remains coherent and defensible; documented the current UI limitations transparently.
- \* **Keeping documentation aligned with real implementation** - Problem: Several documentation files started as templates and could easily drift away from the actual code behavior. - Impact: Risk of thesis documentation being inconsistent with the system (especially around endpoints, error responses, and ?no results? responses). - Resolution: Re-checked controllers/OpenAPI and updated the documentation to match the implemented API responses and the current frontend wiring status.

## Technical Debt & Known Issues

| ID     | Issue                                     | Severity | Description                                                       | Potential Fix                             |
|--------|-------------------------------------------|----------|-------------------------------------------------------------------|-------------------------------------------|
| TD-001 | Frontend routes/pages not fully wired     | High     | Navigation can include internal route configuration and implem... | Finish route configuration and implem...  |
| TD-002 | Inconsistent instantiation of DI usage... | Medium   | Some DI patterns are present but not consistent                   | Standardize one approach (DI containe...  |
| TD-003 | API ?no results? semantics may surpri...  | Low      | Some list endpoints return standard error responses (prefer em... | Standardize error responses (prefer em... |

## Code Quality Issues

- \* Frontend feature pages and router wiring need consolidation (ensure each header link maps to an implemented route and consistent layout state).
- \* Improve end-to-end coverage for primary user flows (browse diseases ? open disease card ? view sources), so regressions are caught early.
- \* Reduce duplication between built JS output (build folder) and TypeScript sources when referencing code in documentation and ensure a single source of truth during development.

## Future Improvements (Backlog)

If there was more time, these features/improvements would be prioritized:

### High Priority

- \* **Finish core frontend flows (MVP completeness)** - Description: Implement and wire the main routes (Home, Sources, Disease details, Research cards) to the real backend endpoints; ensure consistent loading/empty/error states. - Value: Delivers the end-user experience described in the thesis and removes current Not Found gaps. - Effort: Medium-High
- \* **Implement robust localization behavior end-to-end** - Description: Ensure the language switch reliably controls locale in API requests and updates visible content across pages. - Value: Meets the bilingual requirement and improves usability for the target audience. - Effort: Medium

### Medium Priority

- \* **Standardize API response contracts and client handling** - Description: Make list endpoints return consistent shapes (arrays + optional pagination metadata) and normalize behavior in the frontend API client. - Value: Simplifies UI integration and reduces edge-case bugs.
- \* **Strengthen testing for user-facing behavior** - Description: Add Playwright coverage for ?happy path? navigation and empty-state flows; expand backend integration tests for filtering/search. - Value: Prevents regressions and improves confidence during final thesis demos.

Nice to Have

- \* Add basic SEO-friendly SPA metadata (title/description per route) and a sitemap for improved discoverability.
- \* Improve caching strategy (TTL tuning and invalidation plan) for read-heavy endpoints.
- \* Formalize the analysis pipeline (pinned Python env, repeatable scripts, artifact export conventions).

Lessons Learned

Technical Lessons

| Lesson                                   | Context                                  | Application                              |
|------------------------------------------|------------------------------------------|------------------------------------------|
| A normalized schema pays off even for... | Symptoms/risk factors and bilingual c... | Start with a clean relational model a... |
| Standardized error/empty-state contra... | Clients must handle ?no results? and ... | Treat response shapes as part of the ... |
| Reproducible environments reduce proj... | Docker Compose allowed fast setup and... | Containerize early and document the e... |

Process Lessons

| Lesson                                   | Context                                  | Application                              |
|------------------------------------------|------------------------------------------|------------------------------------------|
| Requirements need a freeze point         | Late frontend requirements significan... | Agree on an MVP requirement freeze da... |
| Documentation is easier when written ... | Template docs drift quickly if not up... | Keep docs close to code changes and v... |

What Would Be Done Differently

| Area       | Current Approach                      | What Would Change                          | Why                                      |
|------------|---------------------------------------|--------------------------------------------|------------------------------------------|
| Planning   | Backend + docs stabilized first; S... | Set explicit milestones for ?end Re...     | Prevents a situation where backend is... |
| Technology | SPA + REST + Prisma + Docker          | Keep the stack, but enforce one Redu...    | Reduces maintenance overhead and avoi... |
| Process    | Some requirements clarified late      | Add a formal requirements sign-off Redu... | Reduces last-minute changes and scope... |
| Scope      | Ambitious feature list early          | Cut more aggressively to ?core Redu...     | Reduces the probability of a complet...  |

# Personal Growth

## Skills Developed

| Skill                                    | Before Project        | After Project                            |
|------------------------------------------|-----------------------|------------------------------------------|
| Full-stack system delivery (FE/BE/DB)    | Intermediate          | Advanced (clearer layering, better in... |
| Database modeling + migrations (Prism... | Beginner-Intermediate | Advanced (normalized schema, migratio... |
| Testing + quality practices (Jest/Pla... | Beginner              | Intermediate (foundation in place; cl... |

## Key Takeaways

- \* A coherent MVP requires an end-to-end ?happy path? early, not only strong backend pieces.
- \* Data modeling decisions (translations, junction tables, enums) strongly determine how maintainable the product becomes.
- \* Clear contracts (API responses, empty/error states, and documented limitations) reduce confusion and make thesis evaluation easier.

Retrospective completed: 2026-01-05